Exercise 3

Backpropagation from Scratch

Objective

To understand how gradients are propagated through a neural network using the backpropagation algorithm and how the network weights are updated using gradient descent.

Task 18

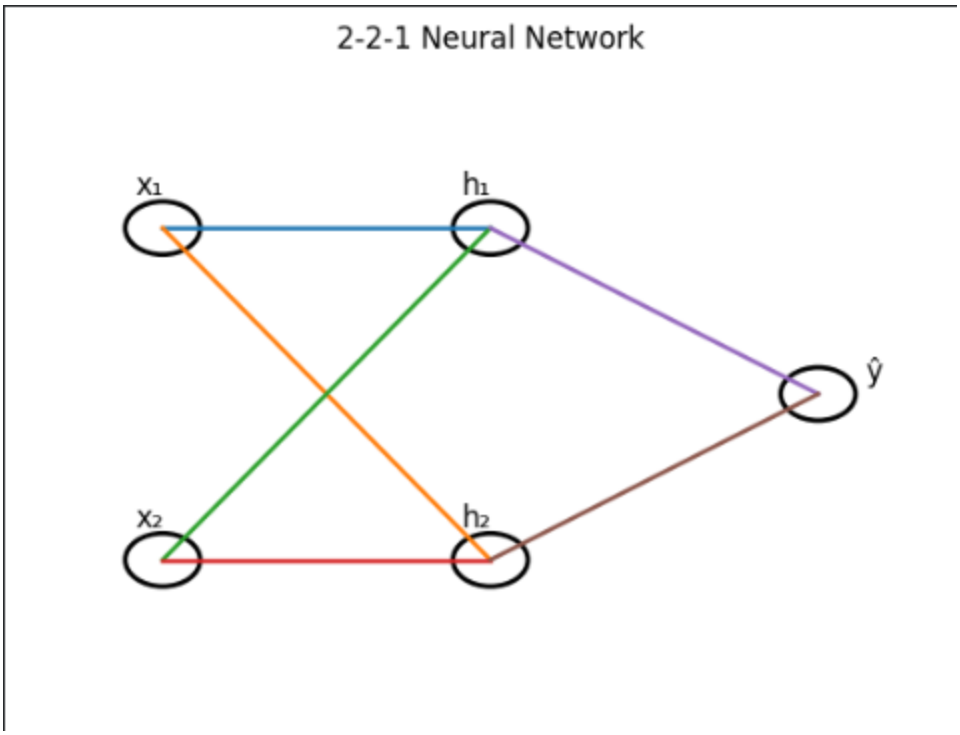
Consider a Network with 2 Input Neurons, 2 Hidden Neurons, and 1 Output Neuron Using Sigmoid Activation and MSE Loss

Theory

A simple feedforward neural network consists of an input layer, a hidden layer, and an output layer. In this exercise, the network contains two input neurons, two hidden neurons, and one output neuron. The hidden and output neurons use the Sigmoid activation function, which produces output values between 0 and 1.

The Mean Squared Error (MSE) loss function is used to measure the difference between the predicted output and the target value. The objective of backpropagation is to minimize this loss by updating the network weights through gradient descent.

The network configuration is given below.



Input

$x=[1,\ 0]$

Target Output

$y=1$

Input-to-Hidden Weights

$W1=[0.2\ 0.4 \quad 0.3\ 0.1]$ *Matrix*

Hidden-to-Output Weights

$W2=[0.5\ 0.6]$ *Matrix*

Task 19

Given Weight Matrices and Input

Theory

The initial weight matrices and input values are used to perform forward propagation through the network. These weights represent the connections between the input layer, hidden layer, and output layer before learning begins.

```
==========================================
 GIVEN PARAMETERS
==========================================

Input Vector
[[1 0]]

Target Value
1

Weight Matrix W1
[[0.2 0.3]
 [0.4 0.1]]

Weight Matrix W2
[[0.5]
 [0.6]]
```

Given

$W1=[0.2\ 0.3 \quad 0.4\ 0.1]$

Input

$x=[1,\ 0]$

Target

$y=1$

Task 20

Perform Forward Propagation

Theory

During forward propagation, the input values are multiplied by the corresponding weights to compute the hidden layer outputs. These outputs are passed through the Sigmoid activation function and then forwarded to the output neuron.

```
==========================================
 FORWARD PROPAGATION
==========================================

Hidden Layer Net Input
[[0.2 0.3]]

Hidden Layer Output
[[0.5498 0.5744]]

Output Layer Net Input
[[0.6196]]

Network Prediction
[[0.6501]]
```

Step 1: Hidden Layer Net Input

$zh=xW1$

$=[1,0][0.2\ 0.4\ \ 0.3\ 0.1\ ] =[0.2,\ 0.3]=[0.2,\|;0.3]=[0.2,0.3]$

Step 2: Hidden Layer Activation

Using

$\sigma(x)=1/1+e^{-x}$

$h1 =\sigma(0.2)=0.5498$

$h2=\sigma(0.3)=0.5744$

Hidden Layer Output

$h=[0.5498, 0.5744]$

Task 21

Compute Hidden Layer Activations and Output Prediction

Theory

The hidden layer outputs become the inputs to the output neuron. The weighted sum is again passed through the Sigmoid activation function to obtain the final prediction.

```
==========================================
 ACTIVATION RESULTS
==========================================

Hidden Activations
[[0.5498 0.5744]]

Predicted Output
[[0.6501]]
```

Output Layer Net Input

$z_o=(0.5498\times0.5)+(0.5744\times0.6)$

$=0.6195$

Output Prediction

$\hat{y} =\sigma(0.6195)$

$\hat{y} =0.6501$

Predicted Output

$\hat{y} =0.6501$

Task 22

Compute the Loss

Theory

The Mean Squared Error loss measures the difference between the predicted output and the target value.

$$Loss = \frac{1}{2}(y - \hat{y})^2$$

Substituting the values,

$$=1/2(1-0.6501)^2$$

=0.0612

```
============================================
 LOSS CALCULATION
============================================

Predicted Output
[[0.6501]]

Mean Squared Error
[[0.0612]]
```

The small loss value indicates that the prediction is reasonably close to the target but still requires weight updates for improvement.

Task 23

Derive Gradients Manually Using Backpropagation

Theory

Backpropagation computes the gradients of the loss function with respect to each weight in the network. These gradients indicate how much each weight contributes to the prediction error and are used to update the weights.

Output Layer Error

$\delta o = (y\hat{} - y) y\hat{} (1 - y\hat{})$

$= (0.6501 - 1)(0.6501)(0.3499)$

$= -0.0796$

Gradient for Output Weights

$$\partial L / \partial W2 = h \times \delta$$

$$= [0.5498 \quad 0.5744\ ](-0.0796)$$

$$= [-0.0438 \quad -0.0457\ ]$$

Hidden Layer Error

$$\delta h = (\delta o W2) \times h(1 - h)$$

$$= [-0.0098, -0.0117]$$

Gradient for Input Weights

Since $x = [1,0]$

the gradients become

$$\partial L / \partial W1 = [-0.0098 \ S \ 0 \quad -0.0117 \quad 0\ ]$$ Matrix

```
============================================
 BACKPROPAGATION
============================================

Gradient of W2
[[-0.043758]
 [-0.045716]]

Gradient of W1
[[-0.009849 -0.011673]
 [ 0.        0.      ]]
```

These gradients are used in the next step to update the weights using gradient descent.

Task 24

Update the Weights Using Gradient Descent

Theory

The weights are updated using Gradient Descent to reduce the prediction error. The update rule is

$$W_{new} = W_{old} - \eta \frac{\partial L}{\partial W}$$

where

Learning Rate

$$\eta = 0.1$$

Updated Hidden-to-Output Weights

$W2 = [0.5044 \quad 0.6046]$ Matrix

Updated Input-to-Hidden Weights

$W1 = [0.2010 \;\; 0.3012 \quad 0.4000 \;\; 0.1000]$

```
...  ============================================
      UPDATED WEIGHTS
     ============================================

     New Weight Matrix W1
     [[0.200985 0.301167]
      [0.4      0.1     ]]

     New Weight Matrix W2
     [[0.504376]
      [0.604572]]
```

The updated weights slightly reduce the loss and improve the prediction during the next forward pass.

Task 25

Verify the Results Using Python

Theory

The forward propagation, backpropagation, and weight update calculations can be verified using Python. The program performs one complete iteration of training and displays the hidden layer activations, output prediction, loss, gradients, and updated weights.

```
============================================================
BACKPROPAGATION VERIFICATION
============================================================

Hidden Layer Input
[[0.2 0.3]]

Hidden Layer Output
[[0.5498 0.5744]]

Predicted Output
[[0.6501]]

Loss
[[0.0612]]

Gradient W1
[[-0.009849 -0.011673]
 [ 0.        0.      ]]

Gradient W2
[[-0.043758]
 [-0.045716]]
```

```
Gradient W2
[[-0.043758]
 [-0.045716]]

Updated W1
[[0.200985 0.301167]
 [0.4      0.1     ]]

Updated W2
[[0.504376]
 [0.604572]]
```

Conclusion

This experiment successfully demonstrated the complete working of the backpropagation algorithm on a simple neural network with two input neurons, two hidden neurons, and one output neuron. Forward propagation was first performed to compute the hidden layer activations, the network prediction, and the Mean Squared Error loss. The gradients of the loss with respect to each weight were then derived manually using the chain rule of calculus, illustrating how the prediction error propagates backward through the network.

Using these gradients, the network weights were updated through gradient descent, resulting in reduced prediction error and improved learning. The manually calculated values were verified using a Python implementation, and the outputs closely matched the mathematical derivation. This experiment provides a clear understanding of how backpropagation enables neural networks to learn by iteratively adjusting weights, making it one of the fundamental algorithms used in training artificial neural networks.